status: implemented date: 2026-05-18 implemented: 2026-05-18 methodology: [MADR] decision-makers: [Henrik Pettersen] tags: [federation, transport, wireguard, upstream-adoption] related: [0097, 0104, 0162, 0186] audience: ai-executor

ADR-0187: Adopt upstream ADR-111 (WireGuard mesh federation layer)?

Context and Problem Statement

ADR-0186's Batch J audit surfaced `70e233946` (upstream "feat(federation): ADR-111 Phases 4-6 — firewall projection + witness chain + MCP tools (#1895)") as a candidate pick. Investigation found this is the **tip of a 4+ commit ADR-111 series** that introduces an opt-in WireGuard mesh layer for federation transport:

Upstream SHA	Date	Subject
`bcdeed8d`	2026-05-10	feat(federation): ADR-111 Phases 1-3 — opt-in WireGuard mesh layer (#1894)
`8f0d90032`	2026-05-10	fix(federation): security — validate peer wg fields before splice (ADR-111)
`70e233946`	2026-05-10	feat(federation): ADR-111 Phases 4-6 — firewall projection + witness chain + MCP tools (#1895)
(possibly more — full chain not enumerated)		

`wg-mesh-service.ts` (the entity the chain hangs off) was never in our fork — we never picked Phases 1-3, so the file doesn't exist for us to modify. Single-pick of `70e233946` is structurally impossible.

Our fork's current federation transport state:

- **ADR-097** (budget envelope + circuit breaker) — landed via `c4175be73` (Phases 2.a/2.b/3-up/4).
- **ADR-104** (wire transport) — landed via the same commit.
- **No ADR-111** (WireGuard mesh).

The architectural question: should we adopt upstream ADR-111's WireGuard mesh layer on top of our existing ADR-097 + ADR-104 transport?

Decision Drivers

- **Surface-area expansion.** ADR-111 adds a substantial new transport layer (WireGuard kernel-level VPN) + firewall + witness chain. This is not a sync mechanic — it's a real product capability decision.
- **Opt-in design.** The upstream PR title says "opt-in WireGuard mesh layer" — adoption may not be load-bearing for default federation paths, but enabling the surface area means maintaining it.
- **Fork divergence cost.** Our ADR-097 / ADR-104 transport is already in production. Stacking ADR-111 on top is feasible but adds maintenance burden when upstream evolves either layer.
- **Use-case justification.** No fork stakeholder has asked for WireGuard-level federation transport; the value of ADR-111 to our consumers is unverified.
- **Security expansion.** WireGuard adds key management, peer validation, firewall projection — real security surface. The `8f0d90032` "validate peer wg fields before splice" fix lands three days after Phases 1-3, suggesting the surface has non-trivial security implications.

Considered Options

1. **Adopt ADR-111 wholesale.** Pick `bcdeed8d` + `8f0d90032` + `70e233946` (+ any later ADR-111 commits at audit time) as a coordinated multi-commit sync, with conflict resolution against our ADR-097 + ADR-104 transport.

2. **Decline adoption.** SKIP all ADR-111 SHAs as `superseded-by-local` on the grounds that our ADR-097 + ADR-104 transport meets current fork needs without the WireGuard surface area.
3. **Defer until user demand.** Mark all ADR-111 SHAs as `pending` and revisit only when a fork consumer specifically asks for WireGuard-level federation transport.

Decision Outcome

Chosen: Option 2 — Decline adoption. No fork consumer has asked for WireGuard-level federation transport; our existing ADR-097 (budget envelope + circuit breaker) and ADR-104 (wire transport) meet current needs. ADR-111's surface area — WireGuard kernel-level VPN + firewall projection + witness chain — carries a maintenance burden disproportionate to its unverified value for our consumers. We revisit only on a concrete consumer request.

SKIP-by-policy rule. Future upstream-sync waves classify all ADR-111-tagged SHAs as `superseded-by-local` referencing ADR-0187. The three currently-known ADR-111 SHAs (`bcdeed8d` , `8f0d90032` , `70e233946`) are flipped from `pending` to `superseded-by-local` in `docs/upstream/INTEGRATION-LEDGER.md` as part of this decision.

Consequences

If adopted (Option 1):

- Multi-commit hand-port effort: 3+ SHAs, conflict resolution against ADR-097 + ADR-104, possible adaptation of `wg-mesh-service.ts` to consume our `federation-coordinator.ts` API.
- Adds runtime dependency on `wg` / `wireguard` system tooling for consumers using the opt-in mesh path.
- Long-term: track upstream ADR-111 evolution in subsequent syncs.

If declined (Option 2):

- No federation feature gap surfaces today.
- Future re-encounter risk: every future upstream sync will list the ADR-111 chain as `pending` ; need a stable SKIP-by-policy rule ("ADR-111 declined per ADR-0187") to avoid re-debating.

If deferred (Option 3 — current state):

- Ledger keeps the 3 ADR-111 SHAs as `pending`.
- Cost: one re-encounter per sync wave until decided.